MES Abasaheb Garware College (Autonomous), Karve Road, Pune

F.Y.B.Sc. (Data Science) Semester-II

DTS-162-PR

Practical course in Data Analysis with R Programming

Work Book

To be implemented from Academic Year 2025–2026

Name: _____

College Name:_____

Roll No.: _____

Division: _____

Academic Year: _____

Introduction

1. About the work book:

This workbook is intended to be used by **F.Y.B.Sc. (Data Science)** students for **Data Analysis with R Programming** Assignments in **Semester–II**. This workbook is designed by considering all the practical concepts / topics mentioned in syllabus.

2. The objectives of this workbook are:

1) Defining the scope of the course.
2) To bring the uniformity in the practical conduction and implementation in all colleges affiliated to SPPU.
3) To have continuous assessment of the course and students.
4) Providing ready reference for the students during practical implementation.
5) Provide more options to students so that they can have good practice before facing the examination.
6) Catering to the demand of slow and fast learners and accordingly providing the practice assignments to them.

3. How to use this workbook:

1) The workbook contains a theory section and assignment questions. Each assignment includes five sub-questions, all of which are compulsory.

4. Instructions to the students

1) Students are expected to carry this book every time they come to the lab for practice.
2) Students should prepare oneself beforehand for the Assignment by reading the relevant material.
3) Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor. However student should spend additional hours in Lab and at home to cover as many problems as possible given in this work book.
4) Students will be assessed for each exercise on a scale from 0 to 5.

Not Done	**0**
Incomplete	**1**
Late Complete	**2**
Needs Improvement	**3**
Complete	**4**
Well Done	**5**

5. Guidelines for Instructors

1) Explain the assignment and related concepts in a round ten minutes if required or by demonstrating the software.
2) You should evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking appropriate box.
3) The value should also be entered on assignment completion page of the respective Lab course.

6. Guidelines for Lab administrator

You have to ensure appropriate hardware and software is made available to each student.

Assignment Completion Sheet

Sr. No.	Assignment Name	Marks (Out of 5)	Signature
1	Introduction to R and Basic Operations		
2	Data Structures and Control Structures in R		
3	Advanced Data Manipulation Techniques in R Recoding Variables		
4	Data Management and Manipulation in R		
5	Data Visualization with ggplot2 in R		
6	Data Aggregation, Cross-Tabulation, Exploring Frequency and Contingency Tables in R		
7	Correlation and Probability Distributions and Performing Univariate Analyses		
8	Working with CSV and Excel Files in R Importing Data from CSV		
9	Introduction to S3 and S4 Classes and Using R Objects and References		
10	Complete Data Analysis Using R- Final Project		
	Total (out of 50)		
	Total (out of 5)		

CERTIFICATE

This is to certify that Mr./Ms._____ has successfully completed **F.Y.B.Sc. (Data Science) DTS-162-PR Practical course in Data analysis with R Programming Semester II** course in year _____and his/her seat no. is _____.

He / She have scored _____ marks out of 15.

Instructor **Internal Examiner** **External Examiner**

ASSIGNMENT NO.1: Introduction to R and Basic Operations

Objective:

The objective of this practical is to familiarize students with the basics of R programming, including getting started with the R environment, understanding fundamental data types and operators, and applying string and numeric functions for basic data processing.

1. Introduction to R

R is an open-source programming language widely used for **data analysis, statistical computing, and visualization**.

It is mainly used by:

- Data Scientists
- Statisticians
- Data Analysts
- Researchers

Key Features of R

- Free and open source
- Powerful statistical analysis
- Large number of packages
- Excellent data visualization
- Works with large datasets

Example: First R Program

```
# First R Program
print("Welcome to R Programming")
```

Output

```
[1] "Welcome to R Programming"
```

2. Basic Operations in R

R can perform **basic mathematical operations** like a calculator.

Arithmetic Operators

Operator	Meaning	Example
+	Addition	5 + 3
-	Subtraction	5 - 3
*	Multiplication	5 * 3
/	Division	5 / 3
^	Power	5 ^ 2

Operator	Meaning	Example
%%	Modulus	5 %% 2

Example

```
a <- 10
b <- 5

# Arithmetic operations
sum <- a + b
difference <- a - b
product <- a * b
division <- a / b
power <- a ^ b
modulus <- a %% b

print(sum)
print(difference)
print(product)
print(division)
print(power)
print(modulus)
```

3. Understanding Data Types in R

Data types define **what type of data a variable can store**.

Major Data Types in R

Data Type	Description	Example
Numeric	Decimal numbers	10.5
Integer	Whole numbers	10L
Character	Text values	"R Programming"
Logical	TRUE or FALSE	TRUE

Example

```
# Numeric
num <- 25.5
print(num)

# Integer
age <- 25L
print(age)

# Character
name <- "Sachin"
print(name)


# Logical
result <- TRUE
print(result)
```

4. Operators in R

Operators are used to **perform operations on variables and values**.

Types of Operators

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators

Example

```
x <- 10
y <- 20

# Relational operators
print(x < y)
print(x > y)

# Logical operators
print(x < y & y > 10)
print(x < y | y < 10)
```

5. String Functions in R

Strings are **text values enclosed in quotes**.

Common String Functions

Function	Purpose
nchar()	Counts number of characters
toupper()	Convert to uppercase
tolower()	Convert to lowercase
paste()	Combine strings
substr()	Extract substring

Example

```
text <- "Data Analysis"

# Length of string
nchar(text)
# Convert to uppercase
toupper(text)

# Convert to lowercase
tolower(text)

# Combine strings
paste("Hello", "World")
```

```
# Extract substring
substr(text, 1, 4)
```

Output

```
13
"DATA ANALYSIS"
"data analysis"
"Hello World"
"Data"
```

6. Numeric Functions in R

Numeric functions are used for **mathematical calculations**.

Common Numeric Functions

Function	Purpose
sqrt()	Square root
abs()	Absolute value
round()	Rounding numbers
ceiling()	Round upward
floor()	Round downward

Example Code

```
x <- 25

# Square root
sqrt(x)              # output : 5

# Absolute value
abs(-10)             # output : 10

# Round value
round(3.567,2)       # output : 3.57

# Ceiling
ceiling(3.2)         # output : 4

# Floor
floor(3.8)           # output : 3
```

Questions:

Q1) Write an R program that:

1. Takes two numeric values from user.
2. Performs **all arithmetic operations** (+, −, *, /, power, modulus).
3. Stores the results in a **vector**.
4. Displays the **maximum, minimum, and average** value of the results.

Q2) Create an R program that:

1. Generates **10 random numbers between 1 and 100**.
2. Calculates **square root and then rounded value, ceiling, and floor** for each square root value.
3. Stores results in a **data frame**.

Q3) Write an R program that:

1. Takes a **list of names**.
2. Converts all names to **uppercase**.
3. Calculates the **number of characters in each name**.
4. Combines the name and its length into a **formatted sentence**.

Q4) Write an R program that:

1. Creates a **vector of numbers from 1 to 200**.
2. Finds numbers that are **divisible by 3 and 5**.
3. Calculates **square root and square** of those numbers.
4. Displays results in a **table format**.

Q5) Write an R program to evaluate a quadratic equation for given a, b, c values and return real or complex roots.

Formula:

For $ax^2 + bx + c = 0$

Discriminant: $D = b^2 - 4ac$

Roots: $x = (-b \pm \sqrt{D}) / 2a$

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

ASSIGNMENT NO.2: Data Structures and Control Structures in R

Objective:

The objective of this practical is to understand various data structures in R such as vectors, lists, matrices, and data frames, and to apply control structures like conditional statements and loops for effective data manipulation and problem solving.

1. Data Structures in R

A **data structure** is a way of organizing data so that it can be processed efficiently.

In data analysis, datasets are stored using different **data structures** depending on the nature of the data.

Major Data Structures in R

Data Structure	Description	Example Use
Vector	Collection of elements of same type	Store sales numbers
Matrix	2D data structure	Store exam marks
List	Mixed data types	Store employee profile
Data Frame	Table format dataset	Store customer data
Factor	Categorical data	Gender, product type

These structures are heavily used in **business analytics, machine learning, and statistical analysis**.

2. Vector

A **vector** is the simplest data structure in R.

Characteristics:

- Stores **same data type**
- One-dimensional
- Supports **vectorized operations**

Vectors are created using the **combine function c()**.

Example: Sales Data

```
sales <- c(1200, 1500, 1800, 2000, 1750)

print(sales)

# Calculate total sales
sum(sales)

# Average sales
mean(sales)
```

3. Matrix

A **matrix** is a **two-dimensional structure** where all elements must be of the same type.

Used for:

- Mathematical operations
- Statistical models
- Image processing

Matrix is created using matrix().

Example: Student Marks

```
marks <- matrix(c(85,90,78,88,92,80), nrow=3)

print(marks)

# Access element
marks[2,1]
```

4. List

A **list** is the most flexible structure in R.

Lists can store:

- Numbers
- Strings
- Vectors
- Data frames

Example: Employee record.

Example

```
employee <- list(
  name = "Rahul",
  age = 30,
  salary = 55000,
  department = "Finance"
)

print(employee)
```

5. Data Frame

A **data frame** is the most important structure used in **data analysis**.

Characteristics:

- Similar to Excel table
- Rows represent observations

- Columns represent variables

Example

```
employees <- data.frame(
  Name = c("Rahul","Amit","Priya"),
  Age = c(30,28,26),
  Salary = c(55000,48000,60000)
)

print(employees)
```

6. Control Structures in R

Control structures allow programs to **make decisions and repeat actions**.

Types of control structures:

Control Structure	Purpose
if	Decision making
if-else	Two condition choices
for loop	Fixed number of iterations
while loop	Run while condition true
break / next	Loop control

These are essential in **data cleaning, filtering, and automation**.

Example: If Condition

```
salary <- 50000

if(salary > 40000){
  print("High Income Group")
}
```

Example: For Loop

```
for(i in 1:5){
  print(i)
}
```

Example: While Loop

```
i <- 1

while(i <= 5){
  print(i)
  i <- i + 1
}
```

7. Data Manipulation in R

Data manipulation means transforming raw data into meaningful information.

Common tasks:

- Filtering rows
- Selecting columns
- Creating new variables
- Sorting data
- Aggregating values

These tasks are fundamental in **data analytics and business intelligence**.

Example

```
sales <- data.frame(
  Product=c("A","B","C","D"),
  Revenue=c(1200,900,1500,1800)
)

# Filter revenue > 1000
sales[sales$Revenue > 1000,]
```

Skill	R Functions Used
Data Import	read.csv()
Data Filtering	[ ]
Aggregation	aggregate()
Sorting	order()
Statistics	mean(), sum(), max()
Categorization	ifelse()
Counting	table()

Questions:

Use following **data frame** to solve following questions

```
set.seed(123)

sales_data <- data.frame(
OrderID = 1:100,
CustomerName =
sample(c("Rahul","Amit","Priya","Neha","Arjun","Sneha","Rohit","Kiran"),100,replace=TRUE),
City = sample(c("Pune","Mumbai","Delhi","Bangalore","Hyderabad"),100,replace=TRUE),
Product = sample(c("Laptop","Mobile","Tablet","Shoes","Watch"),100,replace=TRUE),
Category = sample(c("Electronics","Fashion"),100,replace=TRUE),
Quantity = sample(1:10,100,replace=TRUE),
Price = sample(seq(500,50000,500),100,replace=TRUE),
Discount = sample(c(0,5,10,15,20),100,replace=TRUE),
PaymentMethod = sample(c("Cash","Card","UPI"),100,replace=TRUE)
)

sales_data$Revenue <- sales_data$Quantity * sales_data$Price
```

```
sales_data$DiscountAmount <- sales_data$Revenue * sales_data$Discount / 100
sales_data$FinalAmount <- sales_data$Revenue - sales_data$DiscountAmount

head(sales_data)
```

Q1) Filtering Rows

1. Show orders where Price > 20000.
2. Show orders where City = Pune.
3. Show orders where Quantity > 5.
4. Show orders where Category = Electronics.
5. Combine two conditions.

Q2) Sorting Data

1. Sort dataset by Price.
2. Sort dataset by Revenue.
3. Sort by City alphabetically.
4. Sort by Quantity descending.
5. Sort by multiple columns.

Q3) Aggregation

1. Total revenue by city.
2. Average price by category.
3. Total quantity sold by product.
4. Maximum order value by city.
5. Minimum order value by product.

Q4) Conditional Statements

1. If discount > 10 mark "High Discount".
2. Classify orders into High / Medium / Low value by using if else.
3. Print revenue of each order by using for loop.
4. Calculate cumulative revenue by using while loop
5. Skip fashion category.

Q5) Discount Analysis

1. Find Average discount.
2. Orders with discount > 10%.
3. Total discount amount.
4. City with highest discount.
5. Count discount levels.

Q6) Multi Condition Filtering

1. Electronics + Price > 20000
2. Fashion + Quantity > 5
3. Pune + Discount > 10
4. Revenue > 50000 + Cash payment
5. High price + high quantity

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

ASSIGNMENT NO.3: Advanced Data Manipulation Techniques in R Recoding Variables

Objective:

The objective of this practical is to develop skills in advanced data manipulation in R by recoding variables, creating and sorting new variables, and applying efficient data transformation techniques using the *dplyr* package.

Advanced Data Manipulation in R

Data manipulation is one of the most important steps in **data analysis**. It involves cleaning, transforming, filtering, and restructuring datasets so that meaningful insights can be extracted.

In R, data manipulation can be done using:

- Base R functions
- The **dplyr package**

The **dplyr package** provides a powerful and easy-to-read syntax for manipulating datasets.

Recoding Variables

Recoding means **transforming existing variable values into new categories**.

Example: Converting numeric values into categories like **High / Medium / Low**.

Example

```
sales_data$PriceCategory <- ifelse(sales_data$Price > 30000,
                                   "High",
                                   "Low")
```

Sorting Data

Sorting arranges data in **ascending or descending order**.

Base R Sorting

```
sales_data[order(sales_data$Price), ]
```

Descending Order

```
sales_data[order(-sales_data$Price), ]
```

Computing New Variables

New variables can be calculated using existing columns.

Example:

```
sales_data$Profit <- sales_data$FinalAmount * 0.20
```

dplyr Package for Data Manipulation

The **dplyr package** simplifies data manipulation.

Install and Load

```
install.packages("dplyr")
library(dplyr)
```

Important dplyr Functions

Function	Purpose
filter()	Select rows
select()	Select columns
mutate()	Create new variables
arrange()	Sort data
summarise()	Aggregate data
group_by()	Group data

Example

```
sales_data %>%
  group_by(City) %>%
  summarise(TotalRevenue = sum(Revenue))
```

Questions:

Use following **data frame** to solve following questions

```
set.seed(123)
sales_data <- data.frame(
OrderID = 1:200,
CustomerName =
sample(c("Rahul","Amit","Priya","Neha","Arjun","Sneha","Rohit","Kiran"),200,replace=TRUE),
City = sample(c("Pune","Mumbai","Delhi","Bangalore","Hyderabad"),200,replace=TRUE),
Product = sample(c("Laptop","Mobile","Tablet","Shoes","Watch"),200,replace=TRUE),
Category = sample(c("Electronics","Fashion"),200,replace=TRUE),
Quantity = sample(1:10,200,replace=TRUE),
Price = sample(seq(500,50000,500),200,replace=TRUE),
Discount = sample(c(0,5,10,15,20),200,replace=TRUE),
PaymentMethod = sample(c("Cash","Card","UPI"),200,replace=TRUE)
)

sales_data$Revenue <- sales_data$Quantity * sales_data$Price
sales_data$DiscountAmount <- sales_data$Revenue * sales_data$Discount/100
sales_data$FinalAmount <- sales_data$Revenue - sales_data$DiscountAmount
```

Q1) Problems on Recoding Variables, Sorting and New Variable Creation

a) Create **OrderSize (Small, Medium, Large)**
b) Sort by Quantity descending
c) Sort by City and Revenue
d) Create **Tax column**
e) Create **TotalBill column**

Q2) Problems on dplyr select() and filter()
a) Select CustomerName and Product
b) Select numeric columns
c) Remove Discount column
d) Filter Pune customers
e) Filter multiple conditions (Price > 20000 and Quantity > 5)

Q3) Problems on Using mutate() and arrange()

a) Create Tax variable
b) Create Profit variable
c) Create OrderCategory
d) Create DiscountCategory
e) Create PaymentType variable
f) Sort by Revenue descending by using arrange()

Q4) Problems on Using group_by() and summarise()

a) Total revenue by city
b) Average price by product
c) Total quantity sold by product
d) Maximum revenue by city
e) Minimum revenue by product

Q5) Problems on Customer Analysis

1. Count orders per customer
2. Total revenue per customer
3. Average order value
4. Top customer
5. Sort customers by revenue

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

ASSIGNMENT NO.4: Data Management and Manipulation in R

Objective:

The objective of this practical is to understand and perform data management operations in R, including importing and exporting data, modifying datasets, and performing subset creation, sorting, and aggregation for effective analysis.

Introduction to Data Management in R

Data management is an essential part of **data analysis**. Raw data collected from different sources often contains errors, missing values, and inconsistencies. Therefore, before performing analysis, the data must be **cleaned, organized, and transformed**.

R provides powerful functions and packages for:

- Importing datasets
- Exporting processed data
- Cleaning and modifying data
- Creating subsets
- Sorting and aggregating data

These operations allow analysts to prepare data for **statistical analysis, visualization, and machine learning.**

Importing Data in R

Data can be imported from various formats such as:

- CSV files
- Excel files
- Text files
- Databases

Import CSV File

```
# Import CSV dataset
data <- read.csv("sales_data.csv")

# Display first rows
head(data)
```

Import Excel File

```
# Install package
install.packages("readxl")

library(readxl)

data <- read_excel("sales_data.xlsx")
```

Exporting Data in R

After data processing, results can be exported to external files.

Export CSV File

```
write.csv(data, "output_data.csv", row.names = FALSE)
```

Export Excel File

```
install.packages("writexl")

library(writexl)

write_xlsx(data, "output_data.xlsx")
```

Modifying and Manipulating Data

Data modification includes:

- Adding new variables
- Deleting variables
- Renaming variables
- Transforming variables

Example

```
# Create new column
data$Profit <- data$Revenue * 0.2

# Rename column
names(data)[names(data) == "Revenue"] <- "TotalRevenue"

# Remove column
data$Discount <- NULL
```

Subset Creation

Subsetting means selecting **specific rows or columns** from a dataset.

Example

```
# Select rows where price > 20000
subset(data, Price > 20000)

# Select specific columns
data[, c("CustomerName", "Product")]
```

Sorting Data

Sorting organizes data in ascending or descending order.

```
# Sort by Price
data[order(data$Price), ]

# Sort by Revenue descending
data[order(-data$Revenue), ]
```

Aggregation

Aggregation summarizes data using functions like:

- sum()
- mean()
- max()
- min()

```
aggregate(Revenue ~ City, data, sum)
```

Questions:

Use following data file.

Import dataset from CSV and Excel file

```
sales_data <- read.csv("…\\CSV_File\\sales_data.csv")

sales_data_Excel <- read_excel("…\\Excel_File\\sales_data.xlsx")
```

Q1) Importing and Exploring Data from CSV file and Excel File

a) Import dataset from CSV file and Excel file.
b) Display first 10 rows.
c) Display last 10 rows.
d) Check structure of dataset.
e) Display summary statistics.

Q2) Exporting Data

a) Export dataset to CSV file.
b) Export dataset to Excel file.
c) Export only Electronics category.
d) Export customers from Pune.
e) Export top 20 rows.

Q3) Modifying Data and Subset Creation

1. Create Profit column.
2. Rename FinalAmount column.
3. Remove DiscountAmount column.

4. Add Tax column.
5. Select orders with Quantity > 5 by using subset creation.

Q4) Column Selection

1. Select CustomerName and Product columns.
2. Select first 5 columns.
3. Select numeric columns.
4. Remove Discount column.
5. Select specific columns using index.

Q5) Sorting Data

1. Sort by City alphabetically.
2. Sort by City and Revenue
3. Average price by city.
4. Maximum order value by city.
5. Total quantity sold by city.

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

ASSIGNMENT NO.5: Data Visualization with ggplot2 in R

Objective:

The objective of this practical is to learn data visualization techniques in R using the *ggplot2* package, including creating basic charts and applying advanced visualization methods for better data interpretation.

Introduction to Data Visualization

Data visualization is the graphical representation of data to help understand patterns, trends, and relationships.

Benefits of visualization:

- Makes complex data easier to understand
- Identifies trends and patterns
- Helps decision-making
- Detects outliers and anomalies

R provides many visualization tools, but **ggplot2** is the most powerful and widely used package.

Introduction to ggplot2

The **ggplot2 package** is based on the concept of **Grammar of Graphics**, which means building plots layer by layer.

A typical ggplot structure:

```
ggplot(data, aes(x, y)) + geom_*
```

Components:

Component	Meaning
data	Dataset
aes()	Aesthetic mapping
geom	Type of chart
labs()	Labels
theme()	Styling

Building Basic Charts

Scatter Plot

Used to show relationship between two numeric variables.

```
ggplot(sales_data, aes(x=Price, y=Revenue)) +
```

```
geom_point()
```

Bar Chart

Used for categorical comparison.

```
ggplot(sales_data, aes(x=City)) +
geom_bar()
```

Histogram

Shows distribution of numerical values.

```
ggplot(sales_data, aes(x=Price)) +
geom_histogram(bins=20)
```

Box Plot

Used for identifying outliers and distribution.

```
ggplot(sales_data, aes(x=Category, y=Revenue)) +
geom_boxplot()
```

Line Chart

Used to display trends over time or sequence.

```
sales_data$OrderID <- as.numeric(sales_data$OrderID)

ggplot(sales_data, aes(x=OrderID, y=Revenue)) +
geom_line()
```

Advanced Visualization Techniques

Colored Scatter Plot

```
ggplot(sales_data, aes(x=Price, y=Revenue, color=Category)) +
geom_point()
```

Faceted Charts

Creates multiple charts by category.

```
ggplot(sales_data, aes(x=City)) +
geom_bar() +
facet_wrap(~Category)
```

Customized Chart

```
ggplot(sales_data, aes(x=City, y=Revenue)) +
geom_bar(stat="identity") +
labs(title="Revenue by City",
x="City",
y="Revenue")
```

Real-Life Data Analyst Problems

Problem 1 – Revenue by City

```
library(ggplot2)

city_rev <- aggregate(Revenue~City, sales_data, sum)

ggplot(city_rev, aes(x=City, y=Revenue)) +
geom_bar(stat="identity")
```

Problem 2 – Product Sales Distribution

```
ggplot(sales_data, aes(x=Product)) +
geom_bar()
```

Problem 3 – Revenue vs Price Relationship

```
ggplot(sales_data, aes(x=Price, y=Revenue)) +
geom_point()
```

Problem 4 – Revenue Distribution

```
ggplot(sales_data, aes(x=Revenue)) +
geom_histogram(bins=30)
```

Problem 5 – Profit by Category

```
ggplot(sales_data, aes(x=Category, y=Profit)) +
geom_boxplot()
```

Problem 6 – Product Revenue

```
product_rev <- aggregate(Revenue~Product, sales_data, sum)

ggplot(product_rev, aes(x=Product, y=Revenue)) +
geom_bar(stat="identity")
```

Problem 7 – Quantity Distribution

```
ggplot(sales_data, aes(x=Quantity)) +
geom_histogram()
```

Problem 8 – Revenue Trend

```
ggplot(sales_data, aes(x=OrderID, y=Revenue)) +
geom_line()
```

Problem 9 – Category Comparison

```
ggplot(sales_data, aes(x=Category)) +
geom_bar()
```

Problem 10 – Advanced Scatter Visualization

```
ggplot(sales_data, aes(x=Price, y=Revenue, color=Category)) +
geom_point()
```

Skill	ggplot Function
Scatter Plot	geom_point()
Bar Chart	geom_bar()
Histogram	geom_histogram()
Box Plot	geom_boxplot()
Line Chart	geom_line()
Faceting	facet_wrap()
Customization	labs(), theme()

Advanced Data Visualization with ggplot2 :

1. Heatmaps
2. Correlation Plots
3. Time-Series Visualization
4. Interactive Dashboards
5. Publication-Quality Charts

Sales_Dataset:

```
sales_data <- read.csv("…\\sales_data.csv")
sales_data
```

Heatmaps

Heatmaps display **data intensity using colors** and are useful for **pattern detection**.

Chart 1 – Revenue Heatmap by City

```
library(ggplot2)

city_rev <- aggregate(Revenue ~ City + Category, sales_data, sum)

ggplot(city_rev, aes(City, Category, fill=Revenue)) +
geom_tile() +
scale_fill_gradient(low="yellow", high="red")
```

Chart 2 – Product Sales Heatmap

```
product_sales <- aggregate(Quantity ~ Product + City, sales_data, sum)

ggplot(product_sales, aes(Product, City, fill=Quantity)) +
geom_tile()
```

Chart 3 – Discount Impact Heatmap

```
discount_data <- aggregate(Revenue ~ Discount + Category, sales_data, mean)
```

```
ggplot(discount_data, aes(Discount, Category, fill=Revenue)) +
geom_tile()
```

Correlation Plots

Correlation plots help identify **relationships between variables**.

Chart 4 – Correlation Matrix

```
library(corrplot)

numeric_data <- sales_data[,c("Quantity","Price","Revenue","Profit")]

cor_matrix <- cor(numeric_data)

corrplot(cor_matrix, method="color")
```

Chart 5 – Scatter Correlation Plot

```
ggplot(sales_data, aes(x=Price, y=Revenue)) +
geom_point() +
geom_smooth(method="lm")
```

Chart 6 – Pairwise Correlation

```
library(GGally)

ggpairs(numeric_data)
```

Time-Series Visualization

Time-series charts show **data trends over time**.

Chart 7 – Revenue Over Time

```
daily_rev <- aggregate(Revenue ~ Date, sales_data, sum)

ggplot(daily_rev, aes(Date, Revenue)) +
geom_line()
```

Chart 8 – Profit Trend

```
profit_trend <- aggregate(Profit ~ Date, sales_data, sum)

ggplot(profit_trend, aes(Date, Profit)) +
geom_line(color="blue")
```

Chart 9 – Product Sales Trend

```
product_trend <- aggregate(Quantity ~ Date + Product, sales_data, sum)

ggplot(product_trend, aes(Date, Quantity, color=Product)) +
geom_line()
```

Chart 10 – Category Trend

```
category_trend <- aggregate(Revenue ~ Date + Category, sales_data, sum)

ggplot(category_trend, aes(Date, Revenue, color=Category)) +
geom_line()
```

Advanced ggplot Visualizations

Chart 11 – Bubble Chart

```
ggplot(sales_data, aes(Price, Revenue, size=Quantity)) +
geom_point(alpha=0.6)
```

Chart 12 – Stacked Bar Chart

```
ggplot(sales_data, aes(City, fill=Category)) +
geom_bar()
```

Chart 13 – Density Plot

```
ggplot(sales_data, aes(Revenue)) +
geom_density(fill="blue", alpha=0.4)
```

Chart 14 – Violin Plot

```
ggplot(sales_data, aes(Category, Revenue)) +
geom_violin()
```

Chart 15 – Faceted Charts

```
ggplot(sales_data, aes(Product)) +
geom_bar() +
facet_wrap(~Category)
```

Interactive Visualizations

Interactive charts help users **explore data dynamically**.

Chart 16 – Interactive ggplot using plotly

```
library(plotly)

p <- ggplot(sales_data, aes(Price, Revenue)) +
geom_point()

ggplotly(p)
```

Chart 17 – Interactive Bar Chart

```
city_rev <- aggregate(Revenue~City, sales_data, sum)

p <- ggplot(city_rev, aes(City, Revenue)) +
geom_bar(stat="identity")

ggplotly(p)
```

Dashboard Visualization

Dashboards combine **multiple visualizations in one interface**.

Chart 18 – Basic Dashboard (Shiny)

```
library(shiny)

ui <- fluidPage(
plotOutput("salesPlot")
)

server <- function(input, output) {

output$salesPlot <- renderPlot({
ggplot(sales_data, aes(City)) +
geom_bar()
})

}

shinyApp(ui, server)
```

Publication-Quality Charts

These charts are used for **reports, research papers, and presentations**.

Chart 19 – Minimal Theme

```
ggplot(sales_data, aes(Product, Revenue)) +
geom_boxplot() +
theme_minimal()
```

Chart 20 – Professional Chart Styling

```
ggplot(sales_data, aes(Product, Revenue, fill=Category)) +
geom_bar(stat="identity") +
theme_classic() +
labs(title="Revenue by Product")
```

Chart 21 – Custom Color Palette

```
ggplot(sales_data, aes(Product, Revenue, fill=Category)) +
geom_bar(stat="identity") +
scale_fill_brewer(palette="Set2")
```

Chart 22 – Rotated Axis Labels

```
ggplot(sales_data, aes(Product)) +
geom_bar() +
theme(axis.text.x = element_text(angle=45))
```

Chart 23 – Annotated Chart

```
ggplot(sales_data, aes(Product, Revenue)) +
geom_bar(stat="identity") +
annotate("text", x=2, y=200000, label="High Sales")
```

Advanced Analytical Charts

Chart 24 – Revenue vs Profit

```
ggplot(sales_data, aes(Revenue, Profit)) +
geom_point()
```

Chart 25 – Quantity vs Price Relationship

```
ggplot(sales_data, aes(Quantity, Price)) +
geom_point()
```

Graph Type	Tool
Heatmaps	geom_tile()
Correlation	corrplot()
Time-series	geom_line()
Interactive charts	plotly
Dashboards	shiny
Publication charts	theme_minimal(), theme_classic()

Questions:

Use sales_data.csv to solve following questions

sales_data <- read.csv("…\\sales_data.csv")

Q1) Basic ggplot Charts

a) Create scatter plot of Price vs Revenue
b) Create bar chart for City
c) Create histogram of Price
d) Create boxplot of Revenue by Category
e) Create line chart for Revenue by OrderID

Q2) Customized Visualization

a) Add title to chart
b) Change axis labels
c) Change color of bars
d) Change theme style
e) Rotate x-axis labels

Q3) Multi-Variable Visualization

a) Price vs Revenue colored by Category
b) Price vs Quantity colored by Category
c) Revenue vs Discount
d) Top cities by revenue
e) Top products by sales

Q4) Bar Chart, Scatter Plot, Histogram Analysis

a) Number of orders per city. (Bar Chart).
b) Price vs Revenue. (Scatter Plot)
c) Quantity vs Revenue (Scatter Plot)
d) Distribution of Quantity (Histogram)
e) Histogram by Category (Histogram)

Q5) Box Plot, Line Chart, Faceted Chart Analysis

a) Revenue by Category (Box Plot)
b) Price by Product (Box Plot)
c) Revenue trend by OrderID (Line Chart)
d) Discount trend (Line Chart)
e) City orders by category (Faceted Chart)

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

ASSIGNMENT NO.6: Data Aggregation, Cross-Tabulation, Exploring Frequency and Contingency Tables in R

Objective:

The objective of this practical is to analyze and summarize data using aggregation techniques, apply functions from the apply family, and create frequency and cross-tabulation tables to perform statistical inference.

1. Data Aggregation in R

Data aggregation is the process of summarizing data by applying functions like sum, mean, count, etc., over groups.

Key Functions:

- aggregate()
- tapply()
- by()
- dplyr package (group_by + summarise)

Example:

```
# Sample dataset
data <- data.frame(
Department = c("HR", "IT", "HR", "IT", "Finance"),
Salary = c(30000, 50000, 35000, 60000, 45000)
)

# Aggregate mean salary by department
aggregate(Salary ~ Department, data, mean)
```

2. Apply Family Functions

The apply family helps in performing operations over arrays and lists.

Types:

- apply(): for matrices
- lapply(): for lists
- sapply(): simplified version
- tapply(): grouped operations

Example:

```
matrix_data <- matrix(1:9, nrow=3)

# Row-wise sum
apply(matrix_data, 1, sum)
```

```
# Column-wise mean
apply(matrix_data, 2, mean)
```

3. Frequency Tables

Frequency tables count occurrences of values.

Functions:

- table()
- prop.table()

Example:

```
# Frequency table
gender <- c("M", "F", "F", "M", "M")
table(gender)

# Proportion table
prop.table(table(gender))
```

4. Cross-Tabulation

Cross-tabulation shows relationship between two categorical variables.

Example:

```
data <- data.frame(
Gender = c("M","F","M","F","M"),
Purchase = c("Yes","No","Yes","No","Yes")
)

table(data$Gender, data$Purchase)
```

5. Contingency Tables and Inference

Used for statistical testing such as Chi-Square test.

Example:

```
# Contingency table
tab <- table(data$Gender, data$Purchase)

# Chi-square test
chisq.test(tab)
```

PART 2: DATASET CREATION

Creating CSV File

```
set.seed(123)
```

```
data <- data.frame(
CustomerID = 1:100,
Gender = sample(c("Male", "Female"), 100, replace=TRUE),
Region = sample(c("North", "South", "East", "West"), 100, replace=TRUE),
Purchase = sample(c("Yes", "No"), 100, replace=TRUE),
Amount = sample(1000:10000, 100, replace=TRUE)
)

write.csv(data, "customer_data.csv", row.names=FALSE)
```

Creating Excel File

```
install.packages("writexl")
library(writexl)
write_xlsx(data, "customer_data.xlsx")
```

PART 3: 10 REAL-LIFE PROBLEM STATEMENTS

Problem 1: Retail Sales Analysis

1. Find total sales by region
2. Average purchase amount by gender
3. Frequency of purchases
4. Cross-tab of region vs purchase
5. Perform chi-square test

Solution:

```
data <- read.csv("…\\ data\\CSV File\\customer_data.csv)
aggregate(Amount ~ Region, data, sum)
aggregate(Amount ~ Gender, data, mean)
table(data$Purchase)
table(data$Region, data$Purchase)
chisq.test(table(data$Region, data$Purchase))
```

Problem 2: Customer Behavior Analysis

1. Count customers by gender
2. Mean purchase by region
3. Frequency distribution
4. Cross-tab gender vs purchase
5. Statistical inference

Solution:

```
table(data$Gender)
tapply(data$Amount, data$Region, mean)
table(data$Amount)
table(data$Gender, data$Purchase)
chisq.test(table(data$Gender, data$Purchase))
```

Problem 3: Regional Demand

1. Total purchase amount
2. Region-wise average
3. Frequency table
4. Cross-tab region vs gender
5. Chi-square test

Solution:

```
sum(data$Amount)
tapply(data$Amount, data$Region, mean)
table(data$Region)
table(data$Region, data$Gender)
chisq.test(table(data$Region, data$Gender))
```

Questions:

Use **customer_data.csv** to solve following questions

data <- read.csv("…\\data\\CSV File\\customer_data.csv")

Q1) Sales Performance Analysis

a) Total sales by region
b) Average sales per gender
c) Frequency distribution of purchase
d) Cross-tab: Region vs Purchase
e) Chi-square test

Q2) Customer Segmentation

a) Count customers by region
b) Average purchase by region using tapply
c) Frequency of gender
d) Cross-tab: Gender vs Region
e) Chi-square test

Q3) Purchase Pattern Analysis

a) Total purchase amount
b) Row-wise sum using apply
c) Frequency of purchase
d) Cross-tab: Purchase vs Region
e) Chi-square test

Q4) High-Value Customer Analysis

a) Filter customers with Amount > 5000
b) Mean purchase of filtered group
c) Frequency of region (filtered)
d) Cross-tab: Gender vs Purchase (filtered)
e) Chi-square test

Q5) Business Decision Analysis

a) Total revenue
b) Average revenue per region
c) Frequency distribution of regions
d) Cross-tab: Region vs Purchase
e) Statistical inference

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

ASSIGNMENT NO.7: Correlation and Probability Distributions and Performing Univariate Analyses

Objective:

The objective of this practical is to study relationships between variables using correlation analysis, understand probability distributions, and perform statistical tests such as nonparametric tests, one-sample t-tests, binomial tests, and chi-square goodness-of-fit tests.

Correlation Analysis

What is Correlation?

Correlation measures the **strength and direction of relationship** between two variables.

- If one variable increases and the other also increases → **Positive correlation**
- If one increases and the other decreases → **Negative correlation**
- No pattern → **Zero correlation**

Types of Correlation

1. Pearson Correlation

- Measures **linear relationship**
- Values range from **-1 to +1**

2. Spearman Correlation

- Based on **rank**
- Used when data is **non-linear or ordinal**

3. Kendall Correlation

- Measures **association between ranks**
- More robust for small datasets

R Code

```
cor(x, y, method = "pearson")
cor(x, y, method = "spearman")
cor(x, y, method = "kendall")

cor.test(x, y)  # significance test
```

Interpretation

- r ≈ +1 → strong positive relation
- r ≈ -1 → strong negative relation
- r ≈ 0 → no relation

2. Probability Distributions

What is Probability Distribution?

It describes how values of a variable are **distributed**.

Normal Distribution (Gaussian)

- Bell-shaped curve
- Mean = Median = Mode

Functions:

```
dnorm()  # density
pnorm()  # probability
qnorm()  # quantile
rnorm()  # random numbers
```

Binomial Distribution

- Used for **success/failure experiments**

Example: Tossing a coin

```
rbinom(n, size, prob)
```

Poisson Distribution

- Used for **count data** (events per interval)

Example: number of customers per hour

```
rpois(n, lambda)
```

Interpretation

- Normal → continuous data
- Binomial → fixed trials
- Poisson → rare events

3. Nonparametric Tests

What are Nonparametric Tests?

Used when:

- Data is **not normally distributed**
- Data is **ordinal or skewed**

Types

1. Wilcoxon Test

- Alternative to t-test

```
wilcox.test(x, y)
```

2. Mann-Whitney Test

- Compare two independent groups
  (Same as Wilcoxon in R)

3. Kruskal-Wallis Test

- Compare **more than 2 groups**

```
kruskal.test(value ~ group, data)
```

Interpretation

- p-value < 0.05 → significant difference

4. One-Sample T-Test

Purpose

To test whether sample mean is equal to a known value.

Hypothesis:

- H_0: μ = given value
- H_1: $\mu \neq$ given value

R Code

```
t.test(data, mu = 50)
```

Interpretation

- $p < 0.05 \rightarrow$ reject H_0
- Sample mean is significantly different

5. Binomial Test

Purpose

To test probability of success in binary outcomes.

Example:

- Purchase = Yes/No

R Code

```
binom.test(successes, trials, p = 0.5)
```

Interpretation

- $p < 0.05 \rightarrow$ observed proportion differs from expected

6. Chi-Square Goodness-of-Fit Test

Purpose

To check if observed frequencies match expected distribution.

R Code

```
chisq.test(observed, p = expected)
```

Interpretation

- $p < 0.05 \rightarrow$ distribution is NOT as expected
- $p > 0.05 \rightarrow$ distribution matches expected

🎯 Summary Table (Exam Ready)

Concept	Purpose	Function
Correlation	Relationship between variables	cor(), cor.test()
Normal Distribution	Continuous data	rnorm(), dnorm()
Binomial Distribution	Success/Failure	rbinom()
Poisson Distribution	Count data	rpois()
T-Test	Compare mean	t.test()
Binomial Test	Proportion test	binom.test()
Chi-Square	Frequency comparison	chisq.test()
Nonparametric Tests	Non-normal data	wilcox.test(), kruskal.test()

1. Correlation Analysis

Correlation measures the strength and direction of relationship between two variables.

Types:

- Pearson (linear relationship)
- Spearman (rank-based)
- Kendall (ordinal data)

R Code Example:

```
x <- c(10, 20, 30, 40, 50)
y <- c(15, 25, 35, 45, 60)
```

```
cor(x, y, method = "pearson")
cor(x, y, method = "spearman")
```

2. Probability Distributions

Used to model randomness.

Common Distributions in R:

- Normal: dnorm(), pnorm(), qnorm(), rnorm()
- Binomial: dbinom(), pbinom(), rbinom()
- Poisson: dpois(), ppois(), rpois()

Example:

```
# Normal distribution
rnorm(10, mean=50, sd=10)

# Binomial distribution
rbinom(10, size=10, prob=0.5)
```

3. Nonparametric Tests

Used when data does not follow normal distribution.

Tests:

- Wilcoxon test
- Mann-Whitney U test
- Kruskal-Wallis test

Example:

```
a <- c(10,20,30)
b <- c(15,25,35)

wilcox.test(a, b)
```

4. One-Sample T Test

Tests whether sample mean differs from population mean.

```
data <- c(50,52,48,49,51)
t.test(data, mu=50)
```

5. Binomial Test

Tests probability of success.

```
binom.test(8, 10, p=0.5)
```

6. Chi-Square Goodness-of-Fit Test

Checks whether observed distribution matches expected.

```
observed <- c(20, 30, 50)
expected <- c(0.3, 0.3, 0.4)
chisq.test(observed, p = expected)
```

PART 2: DATASET CREATION

```
set.seed(123)

data <- data.frame(
ID = 1:100,
Age = sample(18:60, 100, replace=TRUE),
Income = sample(20000:100000, 100, replace=TRUE),
SpendingScore = sample(1:100, 100, replace=TRUE),
Purchase = sample(c("Yes","No"), 100, replace=TRUE),
Region = sample(c("North","South","East","West"), 100, replace=TRUE)
)

write.csv(data, "analysis_data.csv", row.names=FALSE)

library(writexl)
write_xlsx(data, "analysis_data.xlsx")
```

PART 3: REAL-LIFE PROBLEMS

Problem 1: E-commerce Income vs Spending Behavior

Scenario: A retail company wants to understand whether higher income leads to higher spending.

1. Compute Pearson correlation between Income and SpendingScore
2. Test significance of correlation
3. Generate normally distributed synthetic income data
4. Perform one-sample t-test (H0: mean income = 50000)
5. Perform binomial test for purchase success rate

Solution:

```
cor(data$Income, data$SpendingScore)
cor.test(data$Income, data$SpendingScore)
rnorm(100, mean(data$Income), sd(data$Income))
t.test(data$Income, mu=50000)
binom.test(sum(data$Purchase=="Yes"), nrow(data), p=0.5)
```

Interpretation:

- Correlation value shows strength of relationship
- p-value < 0.05 ⇒ significant relationship

- t-test checks if income differs from benchmark

Problem 2: Age vs Spending Pattern

Scenario: Company studies whether age influences spending.

1. Pearson correlation
2. Spearman correlation
3. Simulate binomial distribution
4. One-sample t-test on age
5. Chi-square goodness-of-fit for regions

Solution:

```
cor(data$Age, data$SpendingScore)
cor(data$Age, data$SpendingScore, method="spearman")
rbinom(50, 10, 0.5)
t.test(data$Age, mu=30)
chisq.test(table(data$Region))
```

Interpretation:

- Spearman useful for non-linear relation
- Chi-square checks distribution uniformity

Problem 3: Income vs Age Relationship

1. Pearson correlation
2. Kendall correlation
3. Generate Poisson distribution
4. Wilcoxon test
5. Binomial test

Solution:

```
cor(data$Income, data$Age)
cor(data$Income, data$Age, method="kendall")
rpois(50, 5)
wilcox.test(data$Income, data$Age)
binom.test(60, 100, 0.5)
```

Interpretation:

- Kendall used for ordinal relationships
- Wilcoxon used for non-normal data

Problem 4: Marketing Campaign Effectiveness

1. Correlation between income & spending
2. t-test for spending score
3. Binomial test for purchase

4. Chi-square GOF
5. Wilcoxon test

Solution:

```
cor.test(data$Income, data$SpendingScore)
t.test(data$SpendingScore, mu=50)
binom.test(sum(data$Purchase=="Yes"), 100, 0.5)
chisq.test(table(data$Region))
wilcox.test(data$Income, data$SpendingScore)
```

Interpretation:

- Helps evaluate campaign success statistically

Problem 5: Customer Risk Analysis

1. Correlation age-income
2. t-test income
3. Binomial success
4. Chi-square purchase
5. Kruskal-Wallis test

Solution:

```
cor(data$Age, data$Income)
t.test(data$Income, mu=40000)
binom.test(55, 100, 0.5)
chisq.test(table(data$Purchase))
kruskal.test(Income ~ Region, data=data)
```

Interpretation:

- Kruskal test compares multiple groups

Problem 6: Financial Behavior Study

1. Correlation
2. t-test
3. Binomial
4. Chi-square
5. Wilcoxon

Solution:

```
cor(data$Income, data$SpendingScore)
t.test(data$SpendingScore, mu=60)
binom.test(45, 100, 0.5)
chisq.test(table(data$Region))
wilcox.test(data$Age, data$SpendingScore)
```

Interpretation:

- Combines parametric & nonparametric insights

Problem 7: Product Demand Analysis

1. Correlation
2. t-test
3. Binomial
4. Chi-square
5. Kruskal test

Solution:

```
cor(data$Age, data$Income)
t.test(data$Age, mu=35)
binom.test(70, 100, 0.5)
chisq.test(table(data$Region))
kruskal.test(SpendingScore ~ Region, data=data)
```

Problem 8: Sales Probability Study

1. Correlation
2. t-test
3. Binomial
4. Chi-square
5. Wilcoxon

Solution:

```
cor(data$Income, data$SpendingScore)
t.test(data$Income, mu=55000)
binom.test(65, 100, 0.5)
chisq.test(table(data$Purchase))
wilcox.test(data$Income, data$Age)
```

Problem 9: Consumer Pattern Analysis

1. Correlation
2. t-test
3. Binomial
4. Chi-square
5. Kruskal test

Solution:

```
cor(data$Age, data$SpendingScore)
t.test(data$SpendingScore, mu=55)
binom.test(50, 100, 0.5)
chisq.test(table(data$Region))
kruskal.test(Age ~ Region, data=data)
```

Problem 10: Business Decision Model

1. Correlation
2. t-test
3. Binomial
4. Chi-square
5. Wilcoxon

Solution:

```
cor(data$Income, data$Age)
t.test(data$Income, mu=60000)
binom.test(40, 100, 0.5)
chisq.test(table(data$Region))
wilcox.test(data$Income, data$SpendingScore)
```

Use customer_data.csv to solve following questions

```
data <- read_excel("..\\R Practical - 7 Solution\\data\\Excel\\analysis_data.xlsx")
```

Q1) E-commerce Income vs Spending Behavior

Scenario: A retail company wants to understand whether higher income leads to higher spending.

a) Compute Pearson correlation between Income and SpendingScore

b) Test significance of correlation

c) Generate normally distributed synthetic income data

d) Perform one-sample t-test (H0: mean income = 50000)

e) Perform binomial test for purchase success rate

Solution:

a) Compute Pearson correlation between Income and SpendingScore

```
cor(data$Income, data$SpendingScore)
```

b) Test significance of correlation : `cor.test(data$Income, data$SpendingScore)`

c) Generate normally distributed synthetic income data

```
rnorm(100, mean(data$Income), sd(data$Income))
```

d) Perform one-sample t-test (H0: mean income = 50000) : `t.test(data$Income, mu=50000)`

e) Perform binomial test for purchase success rate :

```
binom.test(sum(data$Purchase=="Yes"), nrow(data), p=0.5)
```

Interpretation:

a) Correlation value shows strength of relationship
b) p-value < 0.05 ⇒ significant relationship
c) t-test checks if income differs from benchmark

Q2) Age vs Spending Pattern

Scenario: Company studies whether age influences spending.

a) Pearson correlation

b) Spearman correlation

c) Simulate binomial distribution d) One-sample t-test on age

e) Chi-square goodness-of-fit for regions

Solution:

a) Pearson correlation : cor(data$Age, data$SpendingScore)

b) Spearman correlation: cor(data$Age, data$SpendingScore, method="spearman")

c) Simulate binomial distribution : rbinom(50, 10, 0.5)

d) One-sample t-test on age : t.test(data$Age, mu=30)

e) Chi-square goodness-of-fit for regions : chisq.test(table(data$Region))

Interpretation:

d) Spearman useful for non-linear relation
e) Chi-square checks distribution uniformity

Questions:

Use customer_data.csv to solve following questions

data <- read_excel("..\\R Practical - 7 Solution\\data\\Excel\\analysis_data.xlsx")

Q1) Retail Income vs Spending Strategy

Scenario: A retail company wants to understand whether high-income customers actually spend more.

1. Calculate Pearson correlation between Income and SpendingScore. Interpret the result.
2. Test significance of the correlation. What conclusion can be drawn?
3. Generate a normal distribution using Income and compare with actual data.
4. Perform a one-sample t-test to check if average income is ₹50,000.

5. Test whether purchase success rate differs from 50% using binomial test.

Q2) Customer Segmentation Strategy

Scenario: Business wants to divide customers into meaningful segments.

1. Compute correlation matrix of Age, Income, SpendingScore.
2. Identify strongest relationship and justify.
3. Generate normal distribution for SpendingScore.
4. Perform t-test for SpendingScore = 60.
5. Use Kruskal-Wallis test to compare SpendingScore across regions.

Q3) Financial Risk Assessment

Scenario: Bank wants to assess risk based on income and spending.

1. Analyze correlation between Income and SpendingScore.
2. Test statistical significance.
3. Generate Poisson distribution for number of risky customers.
4. Perform t-test on Income = ₹40,000.
5. Perform binomial test assuming 60% safe customers.

Q4) Product Demand Forecasting

Scenario: Company studies demand patterns based on demographics.

1. Compute Spearman correlation between Age and Income.
2. Explain monotonic relationship if any.
3. Generate binomial distribution for purchase events.
4. Perform t-test on Age = 35.
5. Use chi-square goodness-of-fit for Region distribution.

Q5) Customer Satisfaction Modeling

Scenario: SpendingScore is treated as satisfaction level.

1. Correlate Income and SpendingScore using Pearson.
2. Validate using Spearman correlation.
3. Generate normal distribution for SpendingScore.
4. Perform t-test for mean satisfaction = 55.
5. Apply Wilcoxon test between Age and SpendingScore.

Q6) Sales Probability Analysis

Scenario: Business wants to model probability of purchase success.

1. Convert Purchase into binary and correlate with Income.
2. Generate binomial distribution for purchase trials.
3. Perform binomial test for observed success rate.
4. Perform t-test on Income = ₹55,000.
5. Apply chi-square test for purchase distribution.

Q7) Strategic Business Decision Model

Scenario: Management wants a statistical summary for decision making.

1. Compute correlation matrix among all numeric variables.
2. Identify strongest predictor of SpendingScore.
3. Generate Poisson distribution for simulated demand.
4. Perform t-test on Income benchmark ₹60,000.
5. Perform Kruskal-Wallis test for Age across regions.

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

ASSIGNMENT NO.8: Working with CSV and Excel Files in R
Importing Data from CSV

Objective:

The objective of this practical is to develop the ability to import and export data using CSV files and to work with Excel files in R for efficient data handling and storage.

Working with CSV and Excel Files in R

1. Importing Data from CSV

Introduction

A CSV (Comma-Separated Values) file is a **text file used to store tabular data**, where values are separated by commas. It is one of the most commonly used file formats in data analysis.

Function Used

read.csv()

Syntax

data <- read.csv("file_path", header = TRUE)

Example

data <- read.csv("data.csv")

Important Arguments

- header = TRUE → First row contains column names
- sep = "," → Separator (default is comma)
- stringsAsFactors = FALSE → Prevents automatic factor conversion

Working

- Reads the file from specified path
- Converts it into a **data frame**
- Automatically detects data types

Advantages

- Simple and widely supported
- Easy to import/export
- Lightweight format

Limitations

- Cannot store formatting
- Cannot handle multiple sheets

2. Exporting Data to CSV

Introduction

Exporting data allows saving processed data for future use or sharing.

Function Used

write.csv()

Syntax

write.csv(data, "file_name.csv", row.names = FALSE)

Example

write.csv(data, "output.csv", row.names = FALSE)

Important Arguments

- row.names = FALSE → Prevents row numbers from being saved
- file → Output file path

Working

- Converts data frame into CSV format
- Saves file to specified location

Advantages

- Easy data sharing
- Compatible with Excel and other tools

3. Working with Excel Files in R

Introduction

Excel files (.xlsx) are widely used for storing structured data with multiple sheets and formatting.

Packages Used

- readxl → To read Excel files
- writexl → To write Excel files

Install Packages

```
install.packages("readxl")
install.packages("writexl")
```

Reading Excel File

```
library(readxl)
data <- read_xlsx("data.xlsx")
```

Writing Excel File

```
library(writexl)
write_xlsx(data, "output.xlsx")
```

Multiple Sheets

```
write_xlsx(list(Sheet1 = data, Sheet2 = head(data)), "file.xlsx")
```

Working

- Excel files are read into R as **data frames**
- Supports multiple sheets
- Maintains structured data

Advantages

- Supports multiple sheets
- Better formatting than CSV
- Widely used in business

Limitations

- Requires external packages
- Slightly slower than CSV

Difference Between CSV and Excel

Feature	CSV	Excel
Format	Text	Binary
Sheets	Single	Multiple
Speed	Faster	Slower
Formatting	Not supported	Supported
Size	Smaller	Larger

Notes:

- CSV files are simple text files used for storing tabular data and are handled using read.csv() and write.csv() in R.
- Excel files provide advanced features like multiple sheets and formatting and are handled using packages like readxl and writexl.
- Both formats are essential for data import and export in R programming.

Questions:

Use **data.xlsx** to solve following questions

```
data <- read_excel("..\\R Practical - 8 Solution\\data\\Excel\\data.xlsx", sheet = "sales")
```

Q1) A Retail Sales Analysis (Dataset: sales.csv)

a) Load the sales dataset and display the first 10 records.
b) Calculate **total revenue for each product**.
c) Find the **top 5 products with highest revenue**.

d) Filter sales where **quantity sold is greater than 50**.
e) Find **average product price by category**.

Q2) The E-Commerce Customer Analysis (Dataset: customers.csv)
a) Load the customers dataset and display the first 10 records.
b) Calculate **average purchase amount**.
c) Find customers **above average purchase amount**.
d) Count number of **male and female customers**.
e) Create age groups: Youth (<25), Adult (25–50), Senior (>50)
f) Find **top 10 highest spending customers**.

Q3) The Banking Transaction Analysis (Dataset: transactions.csv)
a) Load the transactions dataset and display the first 10 records.
b) Calculate **total deposits**.
c) Calculate **total withdrawals**.
d) Find accounts with **transactions above 10000**.
e) Calculate **average transaction amount**.
f) Count number of **transactions by type**.

Q4) The Healthcare Patient Dataset (patients.csv)
a) Load the patient's dataset and display the first 10 records.
b) Find patients with **high blood pressure (>140)**.
c) Detect patients with **fever (>37°C)**.
d) Calculate **average patient age**.
e) Find **maximum and minimum blood pressure**.
f) Count number of **patients above 60 years**.

Q5) The Student Performance Dataset (students.csv)
a) Load the students dataset and display the first 10 records.
b) Find students who scored **above 80**.
c) Calculate **average marks by subject**.
d) Find **top scoring student**.
e) Find **students who failed (Marks < 40)**.
f) Count number of students in each subject.

Q6) The Social Media Analytics Dataset (posts.csv)
a) Load the posts dataset and display the first 10 records.
b) Calculate **total engagement per post**.
c) Find posts with **engagement > 500**.
d) Find **most liked post**.
e) Calculate **average engagement**.
f) Find posts with **low engagement (<100)**.

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

ASSIGNMENT NO.9: Introduction to S3 and S4 Classes and Using R Objects and References

Objective:

The objective of this practical is to understand object-oriented programming concepts in R by learning S3 and S4 class systems, exploring class structures and documentation, and working with R objects, memory management, variable referencing, and advanced data structures.

S3 Classes in R
Definition

S3 is a **simple and informal object-oriented system** in R.

- No strict structure
- Based on **naming convention**
- Easy to use

Creating S3 Class

```
person <- list(name="Sachin", age=25)
class(person) <- "Person"
```

Method Definition

```
print.Person <- function(obj){
  cat("Name:", obj$name, "\nAge:", obj$age)
}

print(person)
```

Key Features

- Flexible
- No formal class definition
- Uses **generic functions**

2. S4 Classes in R

Definition

S4 is a **formal and strict OOP system**.

- Requires class definition
- Strong type checking

Creating S4 Class

```
setClass("Person",
        slots = list(
          name = "character",
          age = "numeric"
        ))
```

```
p1 <- new("Person", name="Sachin", age=25)
```

Method Definition

```
setMethod("show", "Person",
        function(object){
          cat("Name:", object@name, "\nAge:", object@age)
        })

p1
```

Key Features

- Strict structure
- Data validation
- Better for large systems

3. Class Reference and Documentation

Inspect Class

```
class(person)
str(person)
```

For S4

```
getClass("Person")
slotNames(p1)
```

4. R Objects and Memory Management

Object Types

- Vectors
- Lists
- Data frames
- Functions

Memory Management

- R uses **Garbage Collection**

```
gc()
```

Remove Object

```
rm(person)
```

5. Referencing Variables

```
x <- 10
y <- x   # copy
```

☞ R uses **copy-on-modify mechanism**

6. Advanced Data Structures

List

```
list(name="A", age=20)
```

Data Frame

```
data.frame(x=1:5, y=6:10)
```

Access Elements

```
data$Age
data[1, ]
```

REAL-LIFE PROBLEMS WITH SOLUTIONS

Use **oop_data.xlsx** to solve following questions

data <- read_excel("..\\R Practical - 9 Solution\\Theory Data Set\\Excel\\oop_data.xlsx")

Q1) Customer Object Modeling (S3)

1. Create S3 class for customer
2. Add attributes (Age, Income)
3. Create print method
4. Access object data
5. Modify object

Solution:

```
cust <- list(name="A", age=30, income=50000)
class(cust) <- "Customer"

print.Customer <- function(x){
  cat(x$name, x$age, x$income)
}

print(cust)

cust$income <- 60000
```

Q2) Employee System (S4)

1. Create S4 class
2. Create object
3. Define method
4. Access slots
5. Validate type

Solution:

```
setClass("Employee",
        slots=list(name="character", salary="numeric"))

e1 <- new("Employee", name="Ravi", salary=50000)
```

```
setMethod("show","Employee",
        function(object){
          cat(object@name, object@salary)
        })

e1
```

Q3) Memory Management

1. Create objects
2. Copy object
3. Modify copy
4. Check memory
5. Remove object

Solution:

```
x <- 1:1000
y <- x

y[1] <- 999

gc()

rm(x)
```

Q4) Data Frame as Object

```
class(data)
str(data)

data$Income
data[1:5,]
```

Q5) List-Based Object

```
obj <- list(a=1, b=2, c=3)
class(obj) <- "MyClass"

obj$a
```

Q6) Advanced Referencing

```
x <- data
y <- x

y$Age[1] <- 99
```

Q7) Class Inspection

```
class(data)
str(data)
attributes(data)
```

Q8) S4 Slot Handling

```
setClass("Product", slots=list(price="numeric"))

p <- new("Product", price=100)
```

p@price

Q9) Mixed Object Handling

```
lst <- list(data, x=1:10)
str(lst)
```

Q10) Object-Oriented Analysis

```
class(data)
summary(data)
```

Concept	Key Insight
S3	Flexible, simple
S4	Strict, safe
Memory	Copy-on-modify
Objects	Reusable
Lists	Advanced structures

Questions:

Use oop_customer_data.xlsx to solve following questions

data <- read_excel("..\\R Practical - 9 Solution\\data\\Excel\\ oop_customer_data.xlsx")

Q1) Customer Object Modeling (S3 System)

Scenario: A retail company wants to convert customer records into structured objects.

1. Create an S3 class "Customer" using dataset rows.
2. Assign attributes like Age, Income, Membership.
3. Create a custom print.Customer() method.
4. Extract and display customer details using object reference.
5. Modify one attribute and observe behavior.

Q2) Advanced Customer Class (S4 System)

Scenario: A banking system requires strict validation of customer records.

1. Define an S4 class "Customer" with slots (Age, Income, Membership).
2. Create objects from dataset rows.
3. Write a show() method.

Q3) Object Referencing and Copy Behavior

Scenario: A data analyst copies dataset for experimentation.

1. Assign dataset to new variable.
2. Modify one column in copied dataset.

3. Check if original dataset changes.
4. Use identical() to compare objects.

Q4) Memory Optimization Study

Scenario: Company wants to optimize memory usage.

1. Create large object from dataset.
2. Check memory usage using object.size().
3. Remove unnecessary objects using rm().
4. Run garbage collection.

Q5) List-Based Customer Object System

Scenario: System uses lists instead of data frames.

1. Convert dataset into list of customers.
2. Assign S3 class to each list element.
3. Access nested values.
4. Modify one customer's data.
5. Print structure using str().

Q6) Class Inspection and Documentation

Scenario: Developer needs to inspect object structure.

1. Check class of dataset.
2. Use str() to inspect structure.
3. Extract attributes.
4. Use summary() for overview.

Q7) S4 Slot Manipulation

Scenario: Product system tracks spending behavior.

1. Create S4 class "SpendingProfile".
2. Add slots for SpendingScore and Income.
3. Create object using dataset values.
4. Access slots using @.
5. Modify slot values and validate.

Q8) Advanced Data Structure Integration

Scenario: Company integrates multiple data sources.

1. Create nested list containing dataset and summary.
2. Access inner elements.
3. Modify nested structure.
4. Apply class to nested object.
5. Display structure using str().

Q9) Object-Oriented Data Analysis

Scenario: Analyst builds reusable object-based analysis system.

1. Convert dataset into S3 object.
2. Create function to calculate average income.
3. Apply function to object.
4. Extend object with new attribute.
5. Validate object consistency.

Q10) Enterprise-Level Object System Design

Scenario: A company wants scalable object-oriented design.

1. Create both S3 and S4 versions of dataset.
2. Compare flexibility vs strictness.
3. Implement method for summary statistics.
4. Document differences between systems.

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

ASSIGNMENT NO. 10: Complete Data Analysis Using R
Final Project

Objective:

To perform a comprehensive data analysis using R programming on a real-world dataset.

Problem Statement:

Download a suitable dataset from the Kaggle platform and conduct a complete data analysis using R programming.

Tasks:

1. Import the dataset into R.
2. Perform data cleaning and preprocessing.
3. Conduct exploratory data analysis (EDA) using appropriate statistical and graphical techniques.
4. Apply relevant analytical methods (e.g., correlation, distributions, hypothesis testing, etc.).
5. Interpret the results obtained from the analysis.
6. Present findings using appropriate visualizations.
7. Write a clear and concise **final conclusion** based on the analysis.

Expected Outcome:

Students will demonstrate the ability to analyze real-world data using R and derive meaningful insights through statistical and graphical methods.

Assignment Evaluation

0: Not Done [ ]
1: Incomplete [ ]
2: Late Complete [ ]
3: Needs Improvement [ ]
4: Complete [ ]
5: Well Done [ ]

Signature of Instructor: _____

End of Lab Workbook

Note for Students:

1. Complete all question sets for each assignment
2. Test your programs thoroughly with different inputs
3. Follow R - Programming naming conventions and coding standards
4. Add comments to explain your code
5. Submit assignments on time.